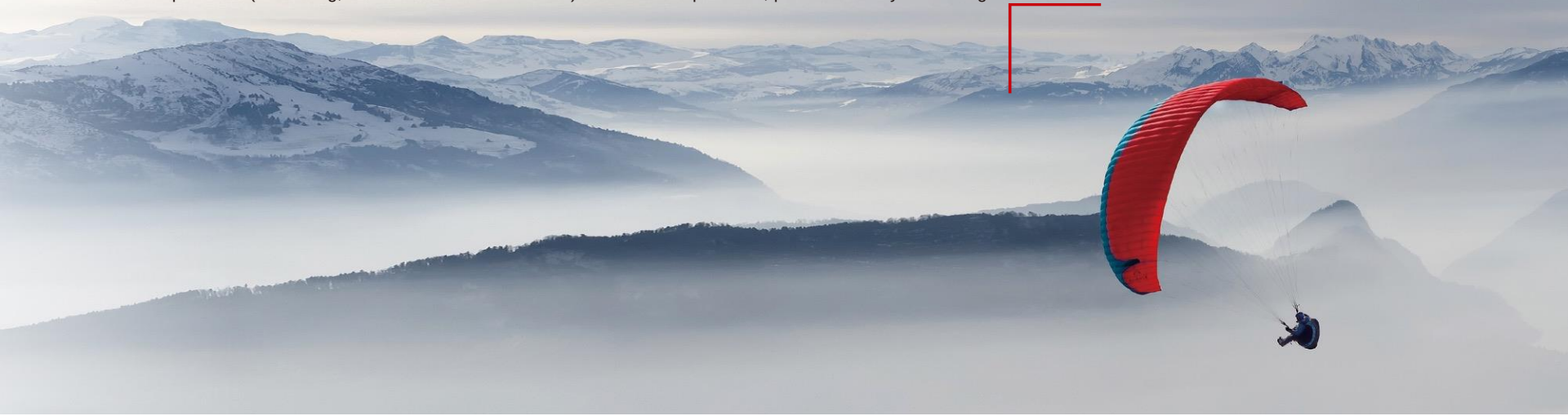
Intelligent Cloud Operations
Part 2. OpenStack Cloud Operating System

Definition (Gartner) [**AIOps**]

AIOps platforms utilize big data, modern machine learning and other advanced analytics technologies to directly and indirectly enhance IT operations (monitoring, automation and service desk) functions with proactive, personal and dynamic insight.



Prof. Jorge Cardoso
E-mail: jorge.cardoso@huawei.com
Large-scale AIOps Lab/SRE Dept.
Munich and Ireland Research Center

07.06.2022

HUAWEI

Intelligent Cloud Operations
The emerging field of AIOps

The field of **AIOps**, also known as **Artificial Intelligence for IT Operations**, uses advanced technologies to dramatically improve the monitoring, operation, and troubleshooting of distributed systems. Its main premise is that operations can be automated using monitoring data to reduce the workload of operators (e.g., SREs or production engineers). Our current research explores how AIOps – and many related fields such as **deep learning, machine learning, distributed traces, graph analysis, time-series analysis, sequence analysis, advanced statistics, NLP and log analysis** – can be explored to effectively **detect**, **localize**, **predict**, and **remediate failures** in **large-scale cloud infrastructures** (>50 regions and AZs) by analyzing **service management data** (e.g., distributed traces, logs, events, alerts, metrics).

- Planet/large-scale Distributed systems and cloud computing
- Distributed traces, logs, events, alerts, metrics, ….
- Big Data platforms with Kubernetes, Hadoop, Spark, Flink, …
- Deep Learning, machine learning, data mining, advanced statistics, time-series analysis, NLP, ….
- Detect, localize, predict and remediate failures in infrastructures

Dr. Jorge Cardoso is Chief Architect for Planet-scale AIOps at Huawei's Ireland and Munich Research Centers. Previously he worked for several major companies such as SAP Research (Germany) on the Internet of Services and the Boeing Company in Seattle (USA) on Enterprise Application Integration. He previously gave lectures at the Karlsruhe Institute of Technology (Germany), University of Georgia (USA), University of Coimbra and University of Madeira (Portugal). His current research involves the development of the next generation of AIOps platforms, Cloud Operations and Analytics tools driven by AI, Cloud Reliability and Resilience, and High Performance Business Process Management systems. He has a Ph.D. in Computer Science from the University of Georgia (USA).

Interests: **AIOps, Service Reliability Engineering, Cloud Computing, Distributed Systems, Business Process Management**

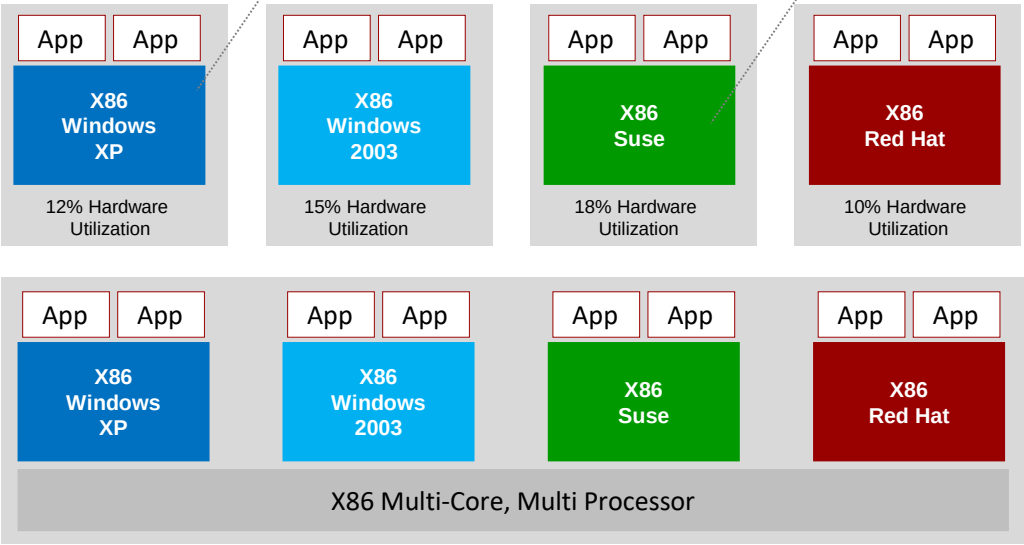
GitHub | Slideshare.net | GoogleScholar

Cloud Computing
Virtualization

Server virtualization is the partitioning of a physical server into multiple smaller virtual servers to maximize resources. The resources of the server are hidden from users.

A virtual machine is an "*efficient, isolated duplicate of a real machine*" (Popek and Goldberg, 1974)

Run the guest operating system as a normal user process on the host. A virtual machine monitor process needs to handle: address space changes, device accesses, system calls,...



70% Hardware Utilization

Popek and Goldberg defined a set of sufficient conditions for a computer architecture to efficiently support virtualization. The set provides guidelines for the design of virtualized computer architectures

Communications of the ACM Vol 17, no 7, 1974, pp.412-421

Formal Requirements for Virtualizable Third Generation Architectures

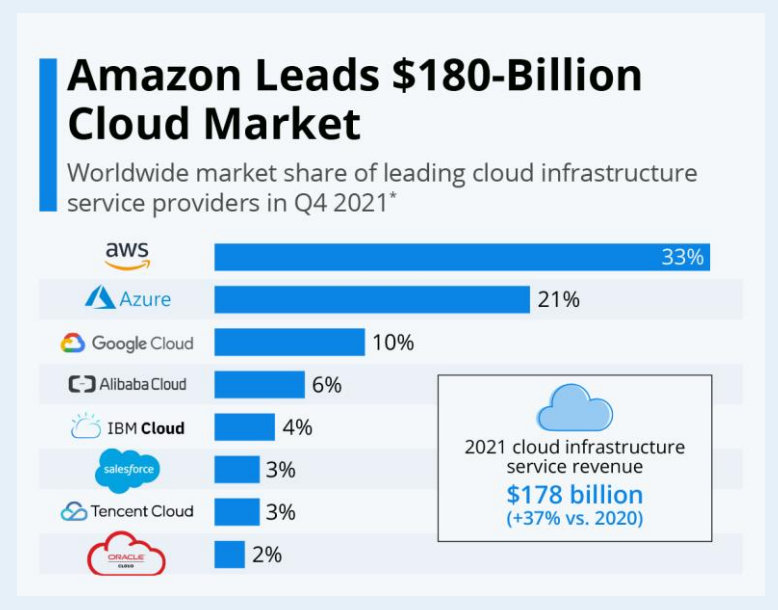
Gerald J. Popek
University of California, Los Angeles
and
Robert P. Goldberg
Honeywell Information Systems and
Harvard University

Virtual machine systems have been implemented on a limited number of third generation computer systems, e.g. CP-67 on the IBM 360/67. From previous empirical studies, it is known that certain third generation computer systems, e.g. the DEC PDP-10, cannot support a virtual machine system. In this paper, model of a third-generation-like computer system is developed. Formal techniques are used to derive precise sufficient conditions to test whether such an architecture can support virtual machines.

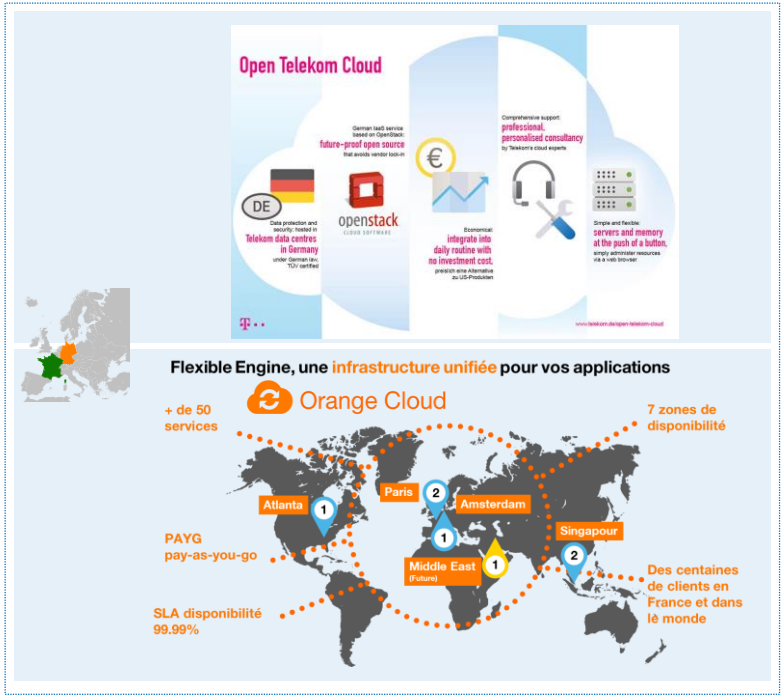
```
$ sudo apt install qemu-kvm libvirt-daemon-system libvirt-
clients bridge-utils virtinst virt-manager
$ virsh create my_vm.xml
$ virsh list
```

Cloud Computing
Public Clouds

- **AWS**, **Azure**, **Google Cloud**, Alibaba Cloud, Huawei Cloud, IBM Cloud, Salesforce, Tencent Cloud, Oracle Cloud, VMWare, etc.
- Transforms datacenters into pools of resources
- Provides a management layer for controlling, automating, and efficiently allocating resources
- Adopts a self-service mode
- Enables developers to build cloud-aware applications via standard APIs



- **Open Telekom Cloud** and **Orange Business Services** offer public cloud services in Germany and France, respectively.



Cloud Computing
OpenStack

- 2010. Joint project of Rackspace Hosting and NASA
- Driven by the emergence of virtualization
- Rackspace wanted to rewrite its cloud servers offering
- NASA had published code for Nova, a Python-based cloud computing controller

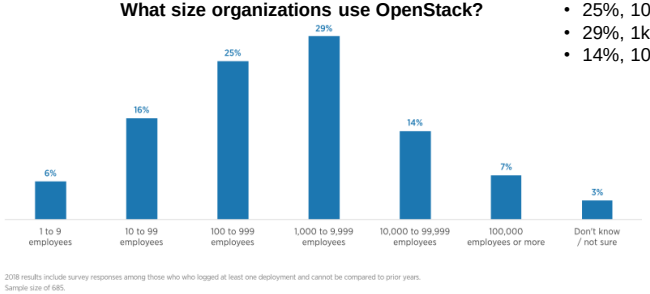
- 2012. Managed by the OpenStack Foundation
- 2018. 500+ companies had joined the project

OpenStack Releases

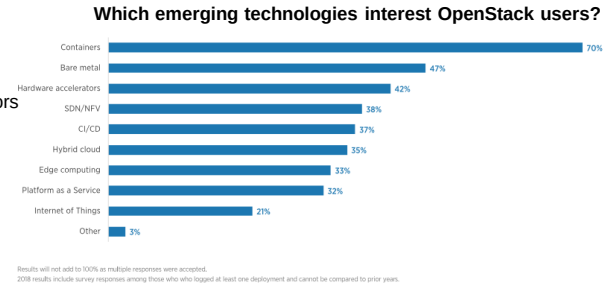
Series	Status	Initial Release Date	Next Phase	EOL Date
Zed	Development	2022-10-05 *estimated* (schedule)	Maintained *estimated 2022-10-05*	
Yoga	Maintained	2022-03-30	Extended Maintenance *estimated 2023-09-30*	
Xena	Maintained	2021-10-06	Extended Maintenance *estimated 2023-04-06*	
Wallaby	Maintained	2021-04-14	Extended Maintenance *estimated 2022-10-14*	
...				
Essex	End Of Life	2012-04-05		2013-05-06
Diablo	End Of Life	2011-09-22		2013-05-06
Cactus	End Of Life	2011-04-15		
Bexar	End Of Life	2011-02-03		
Austin	End Of Life	2010-10-21		

https://releases.openstack.org

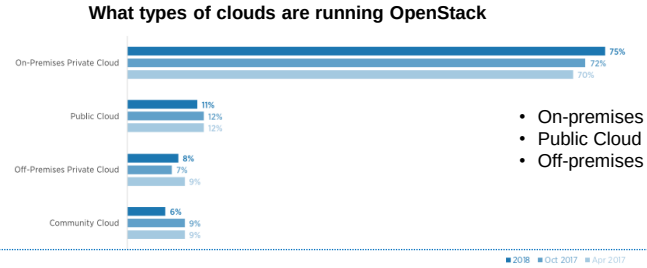
New 2022 OpenStack User Survey will be available this year



- 25%, 100-1k employees
- 29%, 1k-10k employees
- 14%, 10k-100k employees



- Containers
- Base metal
- Hardware accelerators
- SDN/NFV
- CI/CD
- Hybrid Cloud
- Edge Computing
- PaaS



- On-premises Private Cloud
- Public Cloud
- Off-premises Private Cloud

https://www.openstack.org/user-survey/2018-user-survey-report

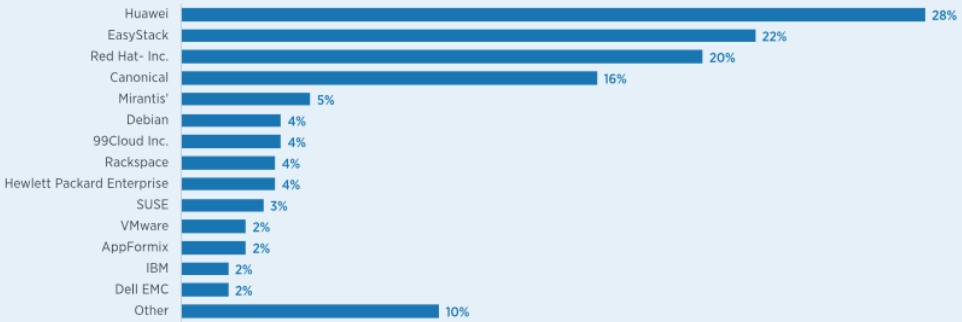
HUAWEI

Cloud Computing
OpenStack Community

- 1,500+ active participants
- 17 countries represented at Design Summit
- 60,000+ downloads
- Worldwide network of user groups (North America, South America, Europe, Asia and Africa)

Key stat: 33% increase in community members YoY

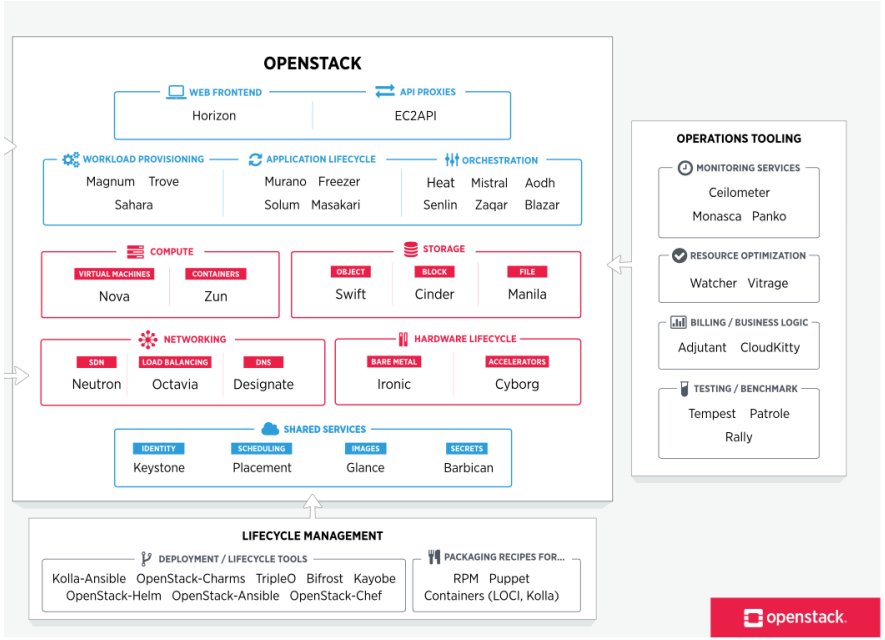
Which vendor's OpenStack-related software products power user's clouds?



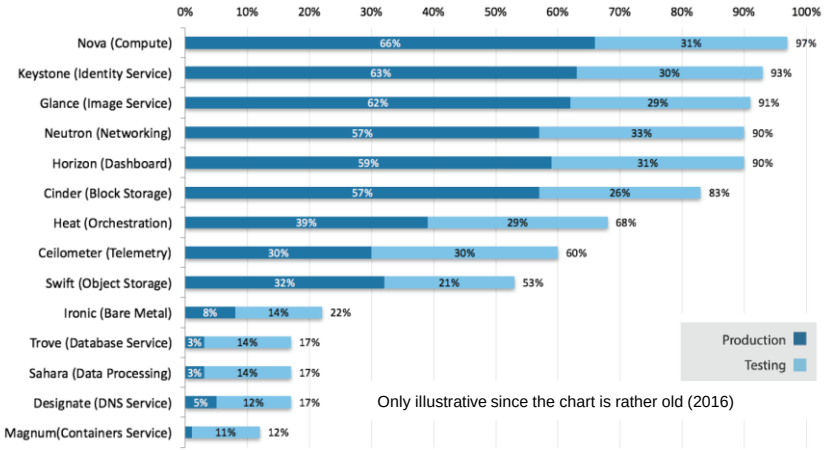
#	Company	Reviews
1	Red Hat	18232
2	Huawei	5348
3	SUSE	4650
4	Rackspace	2948
5	IBM	2644
	*independent	2458
6	Fujitsu	2258
7	NEC	2232
8	Intel	1574
9	AT&T	1344

OpenStack
Popular Services

- OpenStack has many optional plug and play services
- Nova, Keystone, Glance, Neutron, Horizon and Cinder are the ones with greater adoption



https://www.openstack.org/software/



Only illustrative since the chart is rather old (2016)

Additional adoption, maturity and age metrics can be viewed for all of the OpenStack projects at the Project Navigator:
https://www.openstack.org/software/project-navigator

OpenStack
Conceptual Architecture

Lifecycle of creating a VM (OS, CPU, storage, network)

- Horizon. Dashboard
- Keystone. Identity Service
- Glance. Image Service
- Nova. Compute
- Cinder. Block Storage
- Swift. Object Storage
- Neutron. Networking

1. Accessing the dashboard

2. Calling the identity service for authentication
*3. Generating a **token** to be used for subsequent calls*

*4. Contacting the image service to list and retrieve a base image (i.e., an **OS** such as Ubuntu)*

5. Compute service processes the request for a VM

6. Compute service calls the identity service to determine security groups and keys

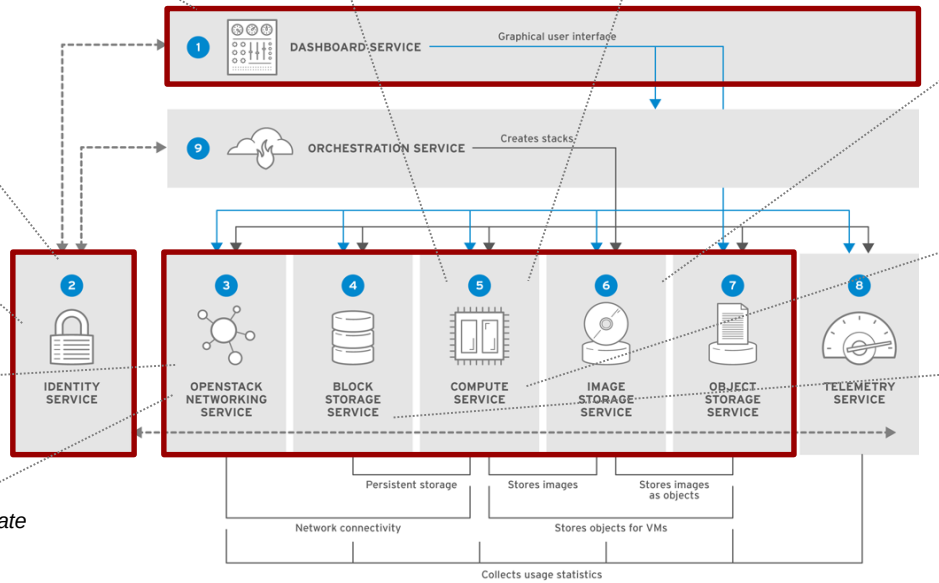
7. Call the network service API to determine available networks

8. Choose the hypervisor node by the compute scheduler service

*9. Call the block **storage** service API to allocate volume to the instance*

*10. Spin up the instance in the hypervisor (i.e., **CPU**) via the compute service API call*

*11. Call the network service API to allocate **network** resources to the instance*

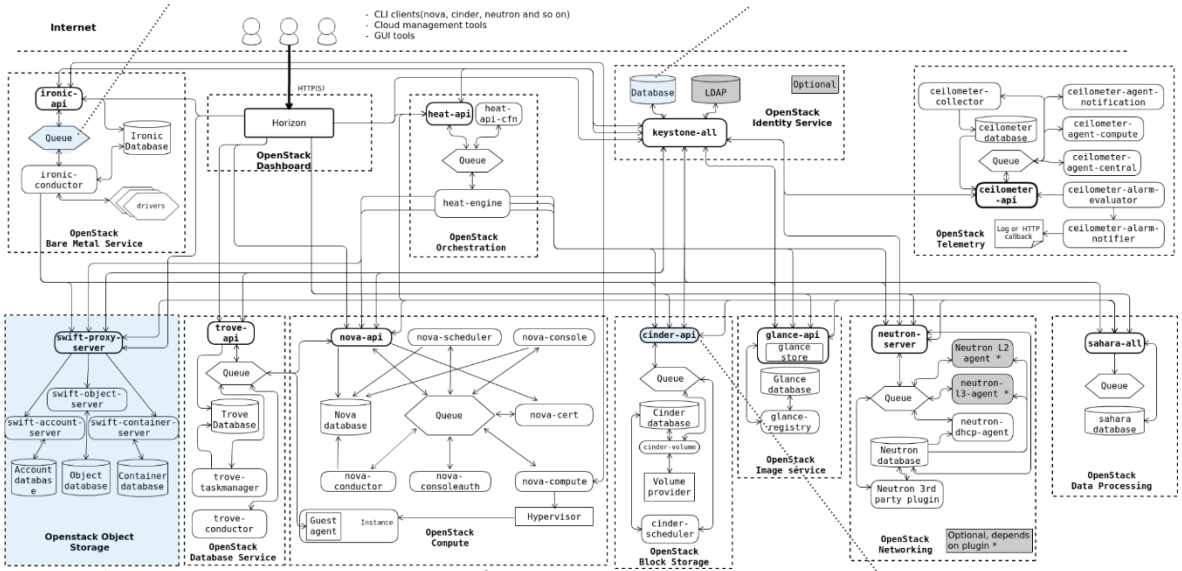


OpenStack
Basic Design Tenets

1. **Scalability** and **elasticity** are the main goals
2. Any feature that limits our main goals must be optional
3. Everything should be **asynchronous**
   - *If you can't do something asynchronously, see #2*
4. All required components must be **horizontally scalable**
5. Always use **shared nothing architecture** or **sharding**
   - *If you can't share nothing/shard, see #2*
6. **Distribute everything**
   - *Especially logic. Move logic to where state naturally exists.*
7. Accept **eventual consistency** and use it where it is appropriate.
8. Test everything
   - *We require tests with submitted code. (We will help you if you need it)*

3 *Message queues provide asynchronous communication and coordination for distributed components*

5 6 *Decentralized data management where each microservice encapsulates its own database*



8 *The system is divided into a number of smaller, individual and independent testable services*

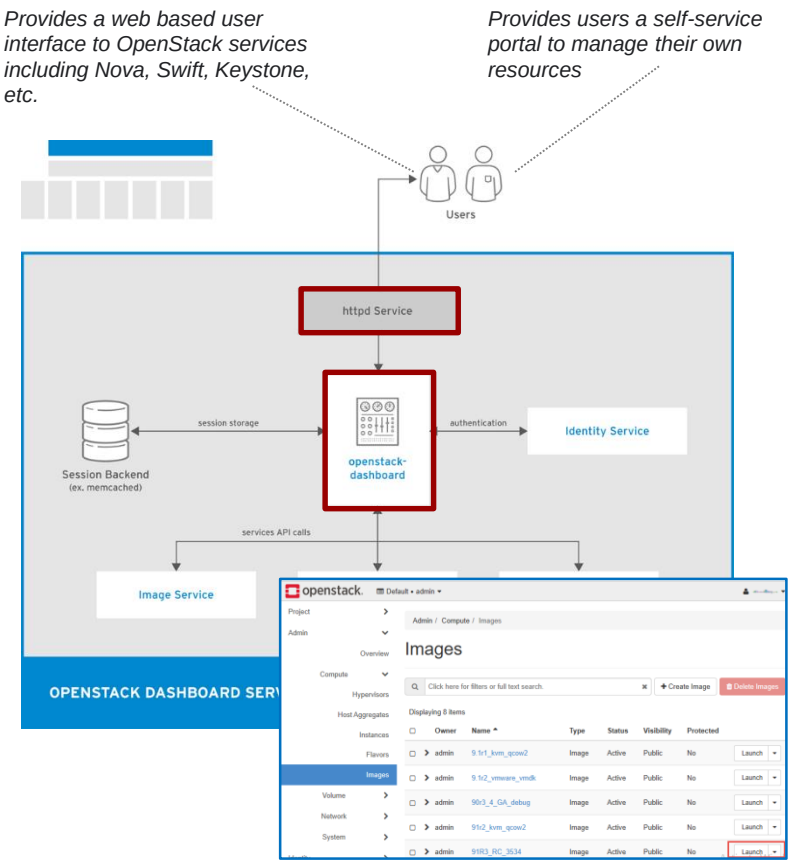
4 *High availability is achieved by expecting that at least 2 servers are utilized to run each service. No single point of failure exists. Service can be replicated to increase scalability*

6 *Each service run as autonomous processes and communicate with one another through APIs*

OpenStack Key Services
Dashboard Service

Key Capabilities

- Thin wrapper over APIs. No local state
- Openstack-dashboard. Django Web application that provides access to the dashboard from any Web browser
- HTTPservice. Apache HTTP server (httpd service)
- Ships with three central dashboards, a “User Dashboard”, a “System Dashboard”, and a “Settings”
- Out-of-the-box support for all core OpenStack projects
  - Nova, Glace, Switch, Neutron, …
- Console Access



OpenStack Key Services
Identity Management Service (IMS)

Key Capabilities

- Provides auth credential validation about Users, Tenants and Roles
- By default, the Identity service uses a MariaDB back end for token, catalog, policy, and identity information
- Token service validates and manages tokens used to authenticate requests after initial credential verification

REST-based APIs

GET	/v3/auth/tokens	Validate and show information for token
HEAD	/v3/auth/tokens	Check token
DELETE	/v3/auth/tokens	Revoke token

Workflow

- Client sends username and password
- Keystone
  - Generates a token
  - Store the token in backend
  - Send a copy of the token back to the client
- Client caches token
- Token is passed along with each API call by the client
- Upon each user request, API endpoint sends token back to Keystone for validation
- Keystone checks token against its auth backend (string and expiration date)
- Keystone returns “success” or “failure” to API endpoint

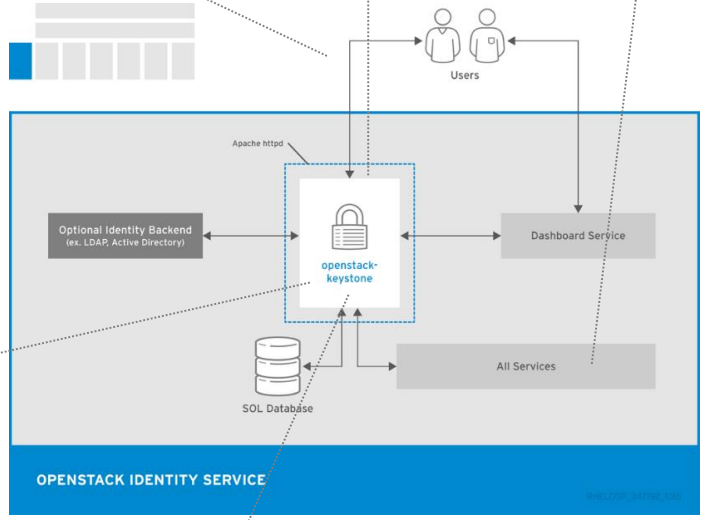
Example of a token

```
$ export TOKEN=`curl --silent -X POST -H "Content-Type: application/json"   -d '{ "auth": {
"identity": { "methods": ["password"], "password": { "user": { "name": "admin", "domain": {
"id": "default" }, "password": "Openstack123" }  } }, "scope": { "project": { "name":
"admin", "domain": { "id": "default" } } } } }' -i "http://192.168.30.10:5000/v3/auth/tokens"
| grep X-Subject-Token | cut -d ":" -f 2`
$ echo $TOKEN gAAAAA...5R-dbCLGBlH-bH7rBIBjh84QhyQtRgqjO-VLvpneBXr....7P9Q
```

The Identity service generates authentication tokens that enable clients to access OpenStack services REST APIs

Clients obtain a token and the URL endpoints for other service APIs by supplying their credentials

The token is often passed within the structure of a larger context of an Identity service response



*Two supported token types: fernet and JWT**

A token can have a variable life span; however the default value for expiry is one hour

(*) https://jwt.io , https://cryptography.io/en/2.8/fernet/

OpenStack Key Services
Compute Service

Key Capabilities

- Manage virtualized server resources
  - CPU, memory, disk, network resources
- Distributed and asynchronous architecture
  - Massively scalable and highly available system

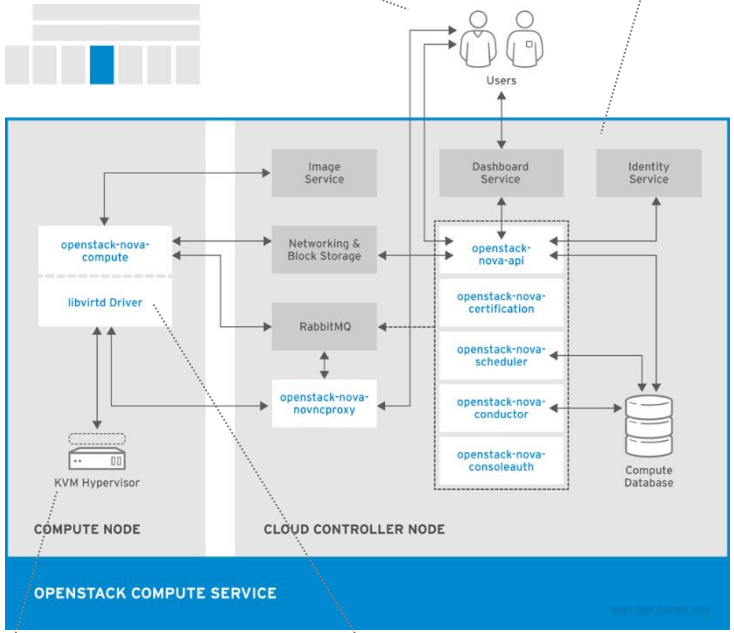
- Comprised of multiple server processes, each performing different functions
- User-facing interface is a REST API, while internal components communicate via an RPC message passing mechanism
- RPC messaging is done using an abstraction on top of message queues via the oslo.messaging library

- Most components can run on multiple servers, and have a manager that is listening for RPC messages
- *Exception*: nova-compute is a single process that runs on the hypervisor it is managing

- To horizontally expand Nova deployments, it uses a deployment sharding concept called cells
  - 2-phase lookup for instance & compute nodes
  - Lookup 3-tuple (database, queue, hostname) for an instance
  - Connect to database and fetch the instance record
  - Connect to the queue and send the message to the proper hostname queue
- Benefits: native sharding, separation between global and local data elements

Nova provides a way to provision compute instances (i.e., virtual machines or servers)

It requires the following additional services:
- *Keystone for identity and authentication*
- *Glance for compute images repository*
- *Neutron for virtual or physical networks*
- *Placement for tracking resources*



https://www.linux-kvm.org https://libvirt.org/index.html

https://access.redhat.com/documentation/en/red-hat-openstack-platform/8/paged/architecture-guide/chapter-1-components
Egan Ford, OpenStack Technology Review & Demo, IBM

OpenStack Key Services
Compute Service: nova-api

Key Capabilities

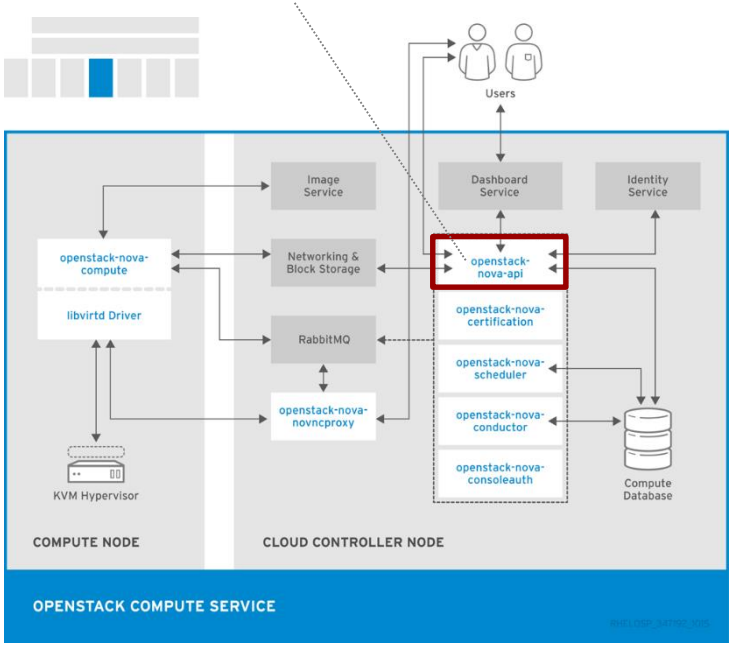
- Provides provisioning and management of virtual compute instances
- Similar in functionality and scope to Amazon's EC2 service via a programmable API
  - default: nova-api listens on port 8773 for the EC2 API and 8774 for the OpenStack API
- nova-api processes REST requests, which typically involve database reads/writes, sending RPC messages to other services, and generating responses to REST calls
- RPC messaging is done using an abstraction on top of message queues via the oslo.messaging library
- Worker daemons carry out orchestration tasks

- Web Server Gateway Interface (WSGI) applications receive API calls

REST-based API

POST	/servers **Create Server**
POST	/servers **Create Multiple Servers**
GET	/servers/detail **List Servers Detailed**
GET	/servers/ `{server_id}` **Show Server Details**
PUT	/servers/ `{server_id}` **Update Server**
DELETE	/servers/ `{server_id}` **Delete Server**

End users and components communicate with nova-api interface to create instances via the OpenStack API or EC2 API



OpenStack Key Services
Compute Service: RabbitMQ and Database

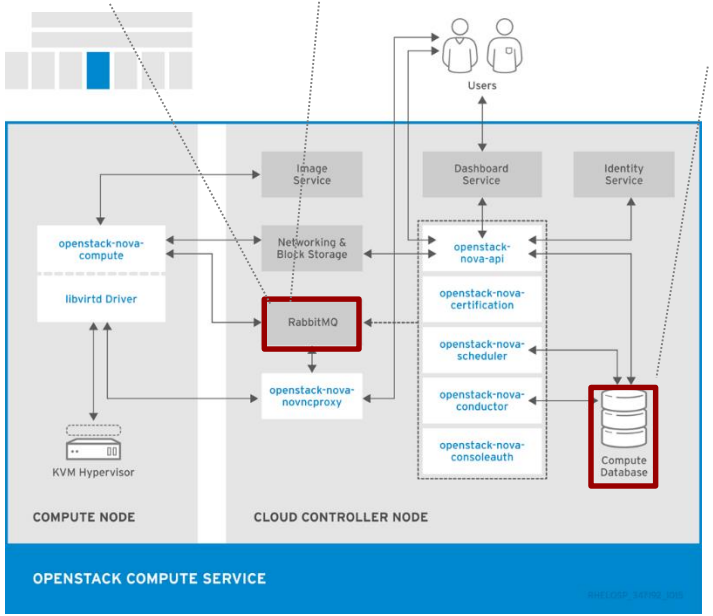
Key Capabilities

- Provides communication hub and manages data persistence
- Asynchronous orchestration between daemons is done through message passing (messaging queue) and information sharing (database)
- Communication through queues
  - Nova creates several types of message queues for communication between daemons
  - API requests are validated and placed on queue
  - Topics queues, fanout queues (advertising of the service capabilities to nova-scheduler workers), and host queues (send messages to specific services on specific hosts)
  - e.g., topics queues allow messages to be broadcast to compute or volume daemons to find available workers to process a message
  - Workers listen to queues based on role or role + hostname
  - Responses are dispatched back through queue
- RabbitMQ is the default the queue
  - ZeroMQ implementation available to decentralize queue
- MySQL DB
  - Single “cell” (1 Queue, 1 Database) typically scales from 500 – 1000 physical machines
  - Stores most of the configuration and run-time state a the cloud infrastructure
  - E.g., instance types are available for use, instances in use, networks available, and projects

Advantages: buffer requests, decoupling, unicast and group-based communication

Message Queue provides a central hub to pass messages between different services in an asynchronous way

Stores build-time and run-time states, including available instance types, instances in use, available networks…

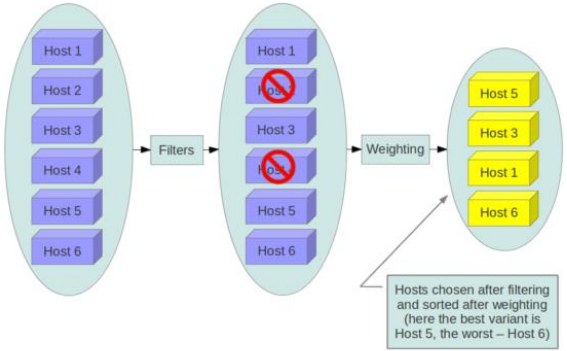


OpenStack Key Services
Compute Service: Scheduler

Key Capabilities

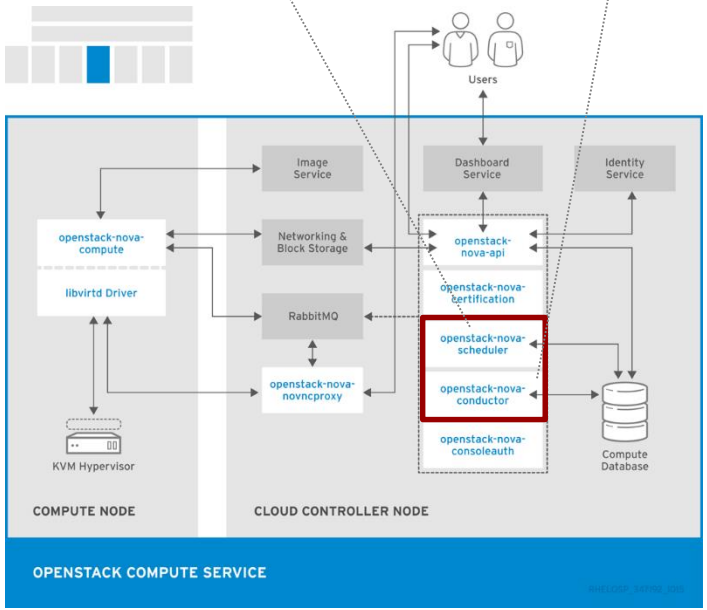
- Compute uses the nova-scheduler to determine how to dispatch compute requests
- Determines which physical hardware to allocate to a VM
- Default scheduler uses a series of filters to reduce set of applicable hosts
  - Have sufficient RAM available (RamFilter)
  - Have sufficient disk space available (DiskFilter)
  - Can service the request (ComputeFilter)
  - Satisfy architecture, hypervisor type, or VM
  - Are on a different host than other instances of a group
  - Are in a set of group hosts
- If default does not work, deployers with specific requirements can develop custom filters

When the scheduler receives a request for a resource, it applies filters to determine which hosts are eligible



nova-scheduler takes a VM instance's request from the queue and determines which compute host it should run on

nova-conductor prevents direct database access from the compute nodes to enhance database security

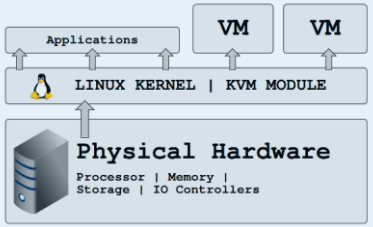


OpenStack Key Services
Compute Service: Compute Node

Key Capabilities

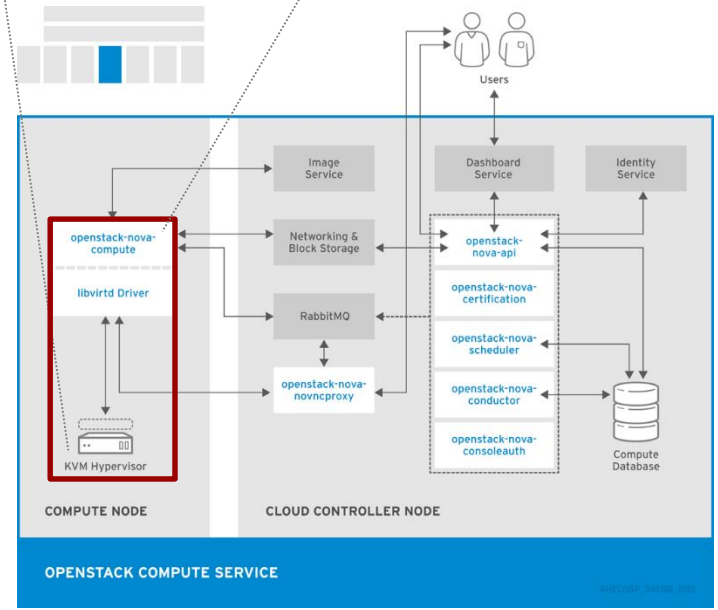
- Worker daemon that creates and terminates VM instances through hypervisor APIs
- Processing is fairly complex
  - daemon accepts actions from the queue and performs a series of system commands such as launching a KVM instance and updating its state in the database
- Nova-compute service is installed on the same node that runs virtual machines
- Service runs on every physical compute node, helps to minimize failure domain
- By default, the selected hypervisor is KVM
  - KVM. Kernel-based Virtual Machine
  - LXC. Linux Containers (through libvirt)
  - QEMU. Quick EMUlator (used for development purposes)
  - VMware vSphere 5.1.0
  - Xen (using libvirt). Xen Project Hypervisor
  - Etc.

KVM/Qemu Virtualization is the foundational technology and the major use scenario in cloud environment.



The majority of nova deployments use libvirt/kvm, but other compute drivers can be used

nova-compute is a daemon that creates and terminates VM instances via the hypervisor's APIs



OpenStack Key Services
Network Service

Key Capabilities

- Neutron creates and attaches interface devices managed by other OpenStack services to networks
  - Attach / detach host to network
- Responsible for managing networks, ports, and attachments on infrastructure for virtual resources
- Common agents are L3 (layer 3), DHCP (dynamic host IP addressing), and a plug-in agent
- Create/delete tenant-specific L2 networks
- L3 support (Floating IPs, DHCP, routing)
- NAT including services such as FWaaS and LBaaS
- Support for
  - Cisco virtual and physical switches
  - NEC OpenFlow products
  - Open vSwitch
  - Linux bridging
  - VMware NSX
  - Cisco Nexus
  - etc.

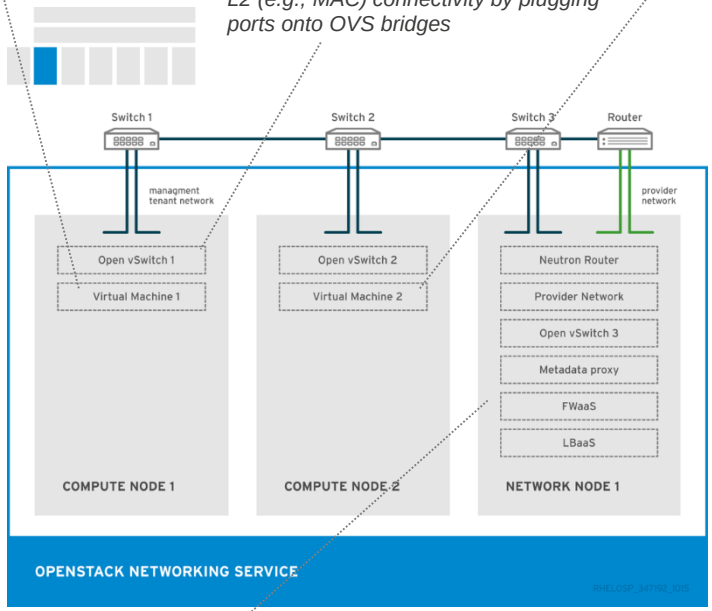
REST-based API

GET	/v2.0/networks/ {network_id}	Show network details
PUT	/v2.0/networks/ {network_id}	Update network
DELETE	/v2.0/networks/ {network_id}	Delete network
GET	/v2.0/networks	List networks
POST	/v2.0/networks	Create network
POST	/v2.0/networks	Bulk create networks

Network as a Service (NaaS) capability between interface devices that are managed by nova

Users can create their own networks and attach server interfaces to them

Open vSwitch (OVS) agent implements L2 (e.g., MAC) connectivity by plugging ports onto OVS bridges



L3 (e.g., IP) agents run only on network nodes and provide routing and NAT services

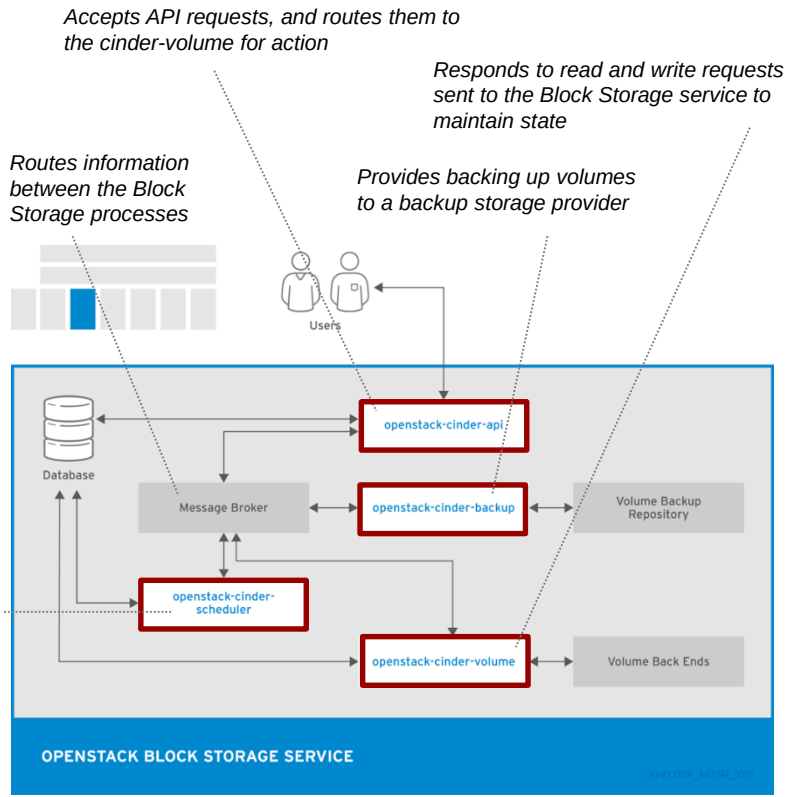
OpenStack Key Services
Block Storage Service

Key Capabilities

- Cinder adds persistent storage to virtual machines
- Responsible for managing lifecycle of volumes and exposing for attachment
- Interacts with OpenStack Compute to provide volumes for instances
- Structure is a copy of Compute (Nova), sharing same characteristics and structure in API server, scheduler, etc.
- Support for booting virtual machines from nova-volume backed storage
- Allows multiple volumes to be attached per virtual machine
- Supports
  - NFS, Ceph distributed file system, etc.

REST-based API

PUT	/v3/ {project_id} /types/ {volume_type_id}	Update a volume type
POST	/v3/ {project_id} /types/ {volume_type_id} /extra_specs	Create or update extra specs for volume type
GET	/v3/ {project_id} /types/ {volume_type_id} /extra_specs	Show all extra specifications for volume type
GET	/v3/ {project_id} /types/ {volume_type_id} /extra_specs/ {key}	Show extra specification for volume type
PUT	/v3/ {project_id} /types/ {volume_type_id} /extra_specs/ {key}	Update extra specification for volume type
DELETE	/v3/ {project_id} /types/ {volume_type_id} /extra_specs/ {key}	Delete extra specification for volume type



OpenStack Key Services
Image Storage Service

Key Capabilities

- Glance enables users to upload and discover data assets used with other services
  - Currently includes images and metadata definitions
- Query for information on public and private disk images
- Register new disk images
- Disk images can be stored in and delivered from a variety of stores (e.g., SoNFS, Swift)
- Supported formats
  - aki/ami/ari (Amazon kernel, ramdisk, or machine image)
  - iso (archive format for optical discs, such as CDs)
  - qcow2 (Qemu/KVM, supports Copy on Write)
  - raw (unstructured format)

Users can add new images or take a snapshot of an existing server for storage

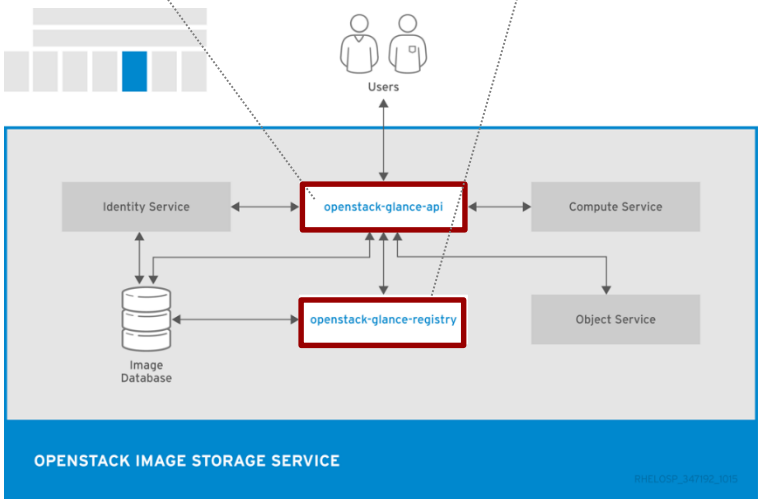
```
$ openstack image list
+--------------------------------------+-------+--------+
| ID                                   | Name  | Status |
+--------------------------------------+-------+--------+
| 38047887-61a7-41ea-9b49-27987d5e8bb9 | cirros | active |
+--------------------------------------+-------+--------+
```

REST-based API

POST	/v2/images	Create image
GET	/v2/images/ {image_id}	Show image
GET	/v2/images/ {image_id} /tasks	Show tasks associated with image
GET	/v2/images	List images
PATCH	/v2/images/ {image_id}	Update image
DELETE	/v2/images/ {image_id}	Delete image
POST	/v2/images/ {image_id} /actions/deactivate	Deactivate image
POST	/v2/images/ {image_id} /actions/reactivate	Reactivate image

Accepts API calls for image discovery, retrieval, and storage

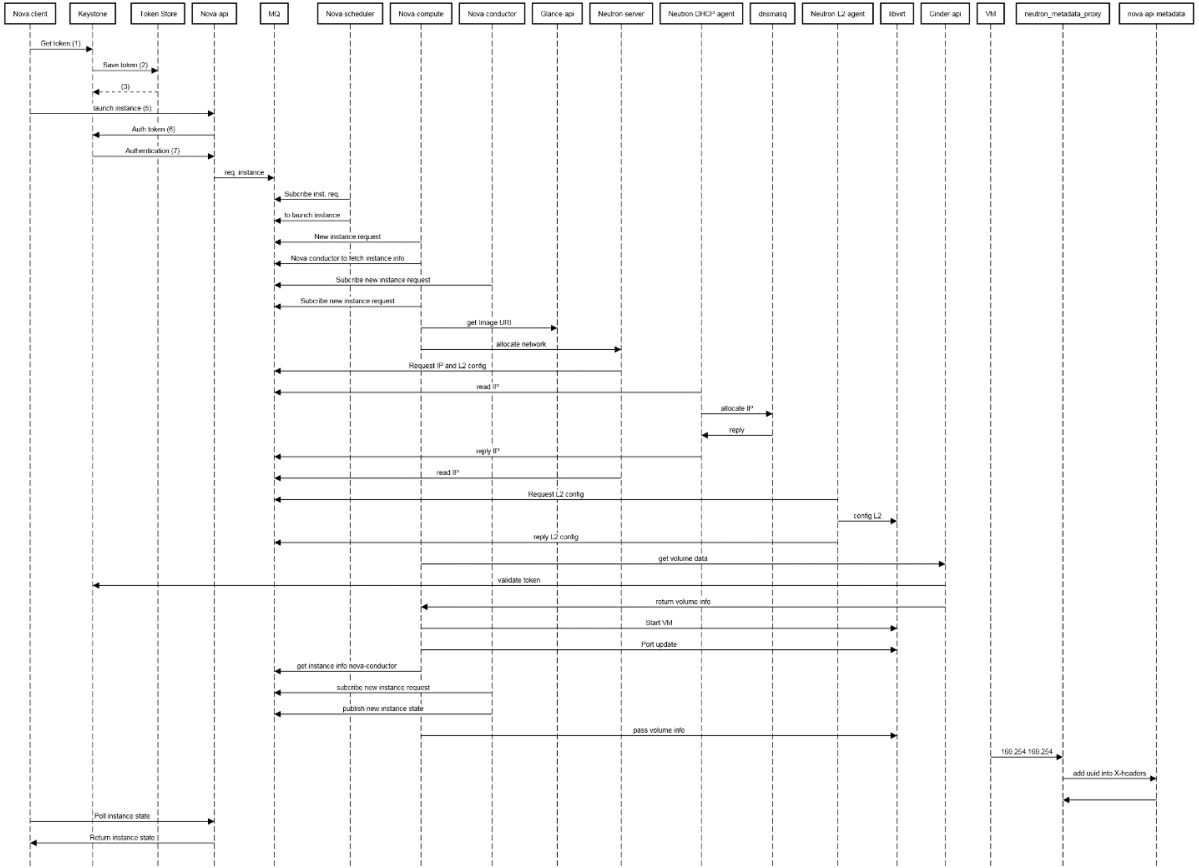
Stores, processes, and retrieves metadata about images. Metadata includes items such as size and type



OpenStack Operation

Server Creation Workflow

```
Nova client-> Keystone: Get token (1)
Keystone-> Token Store: Save token (2)
Token Store--> Keystone: (3)
Keystone-->-Nova client: Auth token (4)

Nova client-> Nova-api: launch instance (5)
Nova-api-> Keystone: Auth token (6)
Keystone->Nova-api: Authentication (7)

Nova-api->MQ: req. instance

Nova-scheduler->MQ: Subscribe inst. req.
Nova-scheduler->MQ: to launch instance

Nova-compute->MQ: New instance request
Nova-compute->MQ: Nova-conductor to fetch instance info
Nova-conductor->MQ: Subscribe new instance request

Nova-compute->MQ: Subscribe new instance request

Nova-compute-> Glance-api: get Image URI
Glance-api->-Nova-compute: Return image URI

Nova-compute->Neutron-server: allocate network
Neutron-server->MQ: Request IP and L2 config

Neutron-DHCP-agent->MQ: read IP
Neutron-DHCP-agent->dnsmasq: allocate IP
dnsmasq->Neutron-DHCP-agent: reply
Neutron-DHCP-agent->MQ: reply IP

Neutron-server->MQ:read IP
Neutron-L2-agent->MQ: Request L2 config
Neutron-L2-agent->libvirt: config L2
Neutron-L2-agent->MQ: reply L2 config
Neutron-server->-Nova-compute: net info

Nova-compute->Cinder-api: get volume data
Cinder-api->Keystone: validate token
Keystone-->-Cinder-api: updated auth headers with roles and acl
Cinder-api->Nova-compute: return volume info

Nova-compute->libvirt: Start VM
Nova-compute->libvirt: Port update
Nova-compute->MQ: get instance info nova-conductor
Nova-conductor->MQ: subcribe new instance request
Nova-conductor->MQ: publish new instance state

Nova-compute->libvirt: pass volume info

VM->neutron_metadata_proxy: 169.254.169.254
neutron_metadata_proxy->nova-api-metadata: add uuid into X-headers
nova-api-metadata->neutron_metadata_proxy:
neutron_metadata_proxy->-VM-instance: return metadata

Nova client->Nova-api: Poll instance state
Nova-api-> Nova client: Return instance state
```



https://sequencediagram.org/

OpenStack
DevStack Deployment

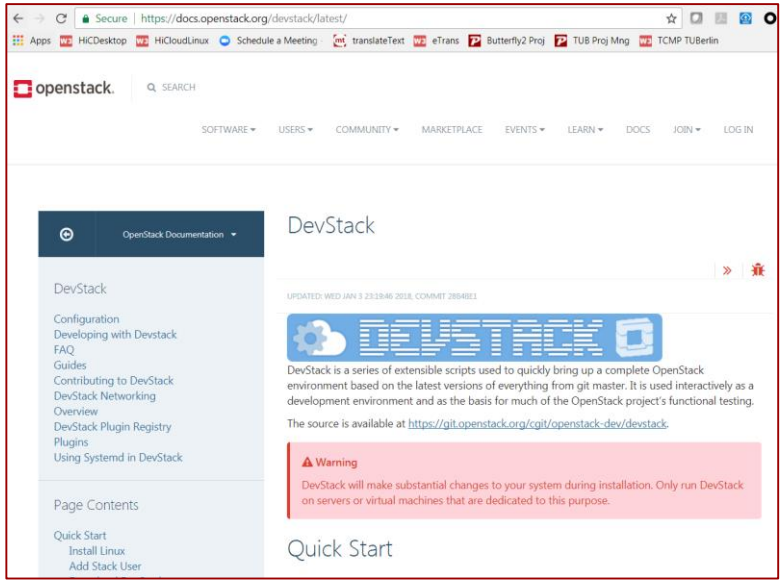
1 Create a VM at LRZ TU Munich

16 GB, image: Ubuntu_20G, 1 vCPU, (optional: access to the Internet)

- Generate ssh key pairs: lrz and lrz.pub
- Copy the public key lrz.pub in the field SSH_PUBLIC_KEY
- Take a note of the public IP

Access the LRZ VM

- ssh-keygen -R 141.40.254.130 (in case you rebuild your VM)
- ssh -i lrz -v root@141.40.254.130



2 Install Openstack

Follow the instructions at https://docs.openstack.org/devstack/latest/

Add Stack User

- $ sudo useradd -s /bin/bash -d /opt/stack -m stack
- $ echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
- $ sudo su - stack

Download DevStack¶

- $ git clone https://git.openstack.org/openstack-dev/devstack
- $ cd devstack

Create a local.conf

Start the install

- ./stack.sh

... wait 15 - 20 minutes …

Inspecting the services installed

- sudo systemctl status "devstack@*"

Inspecting the code of the services

- ls –al /opt/stack/

Inspecting the dashboard (Horizon)

- http://141.40.254.130/
- User Name: admin
- Password: admin (specified in the file local.conf)

https://docs.openstack.org/install-guide/get-started-logical-architecture.html

OpenStack
RDO Deployment

https://www.rdoproject.org/install/quickstart/

If you are using non-English locale make sure your /etc/environment is populated:

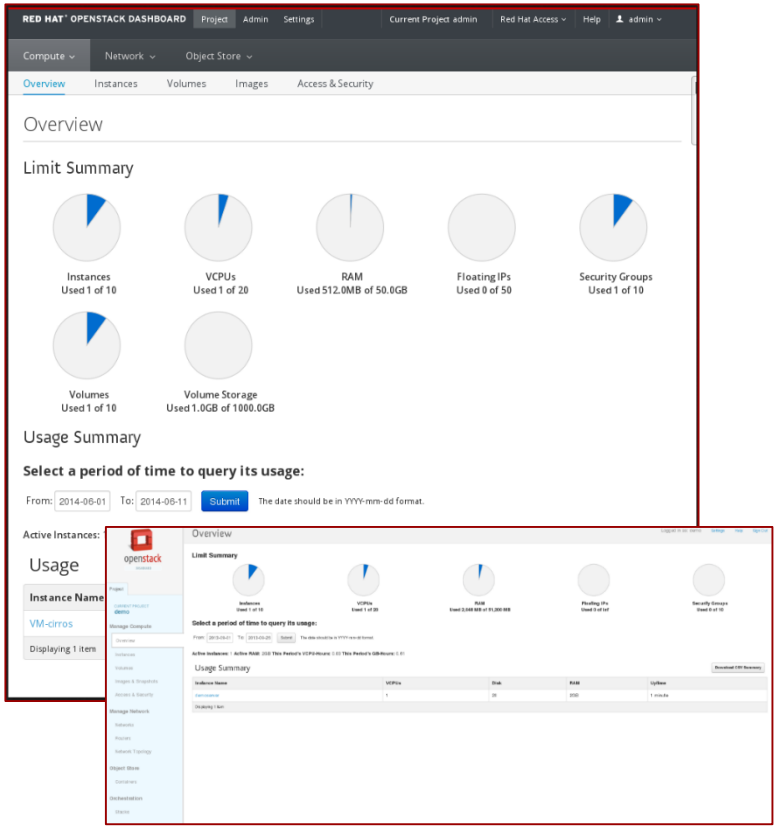
- LANG=en_US.utf-8
- LC_ALL=en_US.utf-8

On RHEL

```
$ sudo yum install -y https://www.rdoproject.org/repos/rdo-release.rpm
$ sudo yum update –y
$ sudo yum install -y openstack-packstack
$ sudo packstack --allinone
```

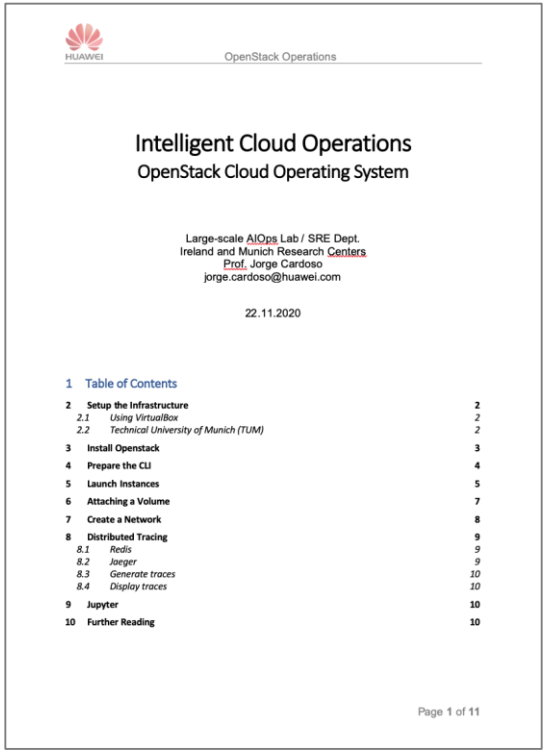
On CentOS

```
$ sudo yum install -y centos-release-openstack-pike
$ sudo yum update –y
$ sudo yum install -y openstack-packstack
$ sudo packstack --allinone
```



OpenStack
Hands-on Exercises

- The following document contains the exercises for this lecture
  - Setup the Infrastructure
  - Install Openstack
  - Prepare the CLI
  - Launch Instances
  - Attaching a Volume
  - Create a Network



HUAWEI OpenStack Operations

Intelligent Cloud Operations

OpenStack Cloud Operating System

Large-scale AIOps Lab / SRE Dept.
Ireland and Munich Research Centers
Prof. Jorge Cardoso
jorge.cardoso@huawei.com

22.11.2020

1 Table of Contents

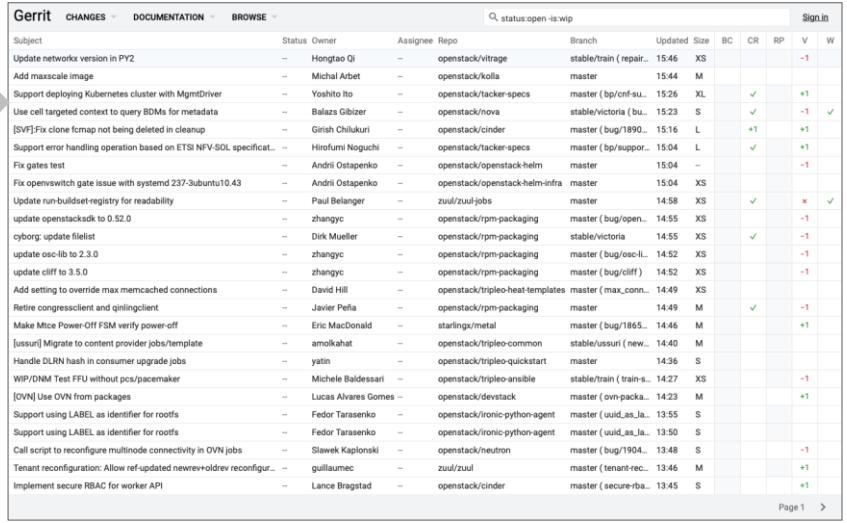
2 Setup the Infrastructure 2
2.1 Using VirtualBox 2
2.2 Technical University of Munich (TUM) 2
3 Install Openstack 3
4 Prepare the CLI 4
5 Launch Instances 5
6 Attaching a Volume 7
7 Create a Network 8
8 Distributed Tracing 9
8.1 Redis 9
8.2 Jaeger 9
8.3 Generate traces 10
8.4 Display traces 10
9 Jupyter 10
10 Further Reading 10

Page 1 of 11

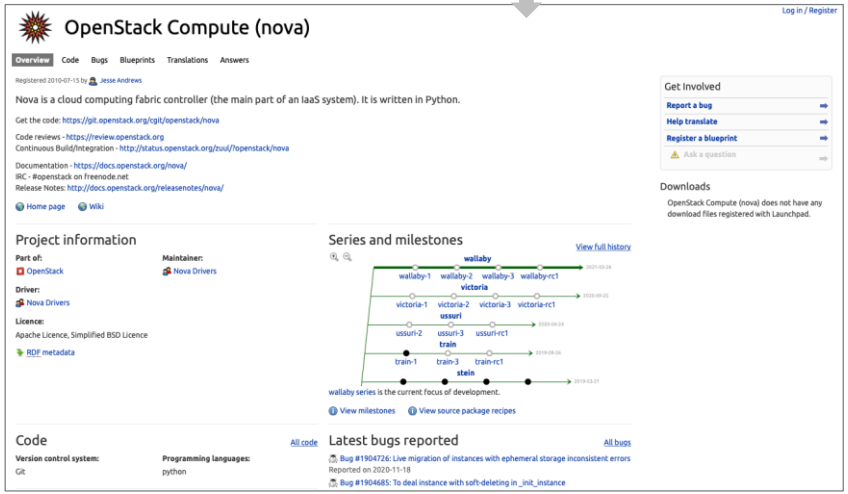
OpenStack
Resources

- **Website:** http://openstack.org
- **Q&A**: https://ask.openstack.org/en/questions/
- **Wiki**: http://wiki.openstack.org/
- **Documentation**: http://docs.openstack.org/
- **Code Review**: https://review.openstack.org/
- **Mailing Lists**: http://wiki.openstack.org/MailingLists
- **Projects**: https://launchpad.net/openstack
- **Blogs**: http://planet.openstack.org
- **Real-time chat room**: #openstack and #openstack-dev on irc://freenode.net (443 users currently logged in)

Code Review



Projects



Thank you.

Bring digital to every person, home and organization for a fully connected, intelligent world.

Copyright©2019 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI